Node.js Basic Interview Questions

1. What is Node.js?

Answer: Node.js is an open-source, cross-platform runtime environment that allows the execution of JavaScript code outside a browser. It is built on Chrome's V8 JavaScript engine and is primarily used for developing server-side and networking applications.

2. Why is Node.js Single-Threaded?

Answer: Node.js operates on a single-threaded event loop model, enabling it to handle multiple client requests efficiently without creating multiple threads. This design is particularly effective for I/O-bound applications.

3. What is the use of the Event Loop in Node.js?

Answer: The Event Loop handles asynchronous callbacks in Node.js. It allows non-blocking I/O operations by delegating tasks to the system kernel whenever possible and executing callbacks after tasks are complete.

4. What are the major advantages of using Node.js?

Answer:

- Asynchronous and Event-Driven architecture
- Fast Execution with Chrome's V8 engine
- Unified JavaScript Development (both frontend and backend)
- Extensive ecosystem with npm modules

5. What is npm?

Answer: npm stands for Node Package Manager. It is the largest software registry that provides access to reusable packages and tools for dependency management in Node.js applications.

Related Read: Coding and Programming Interview Questions [[curent_year]]

6. What is the difference between Node.js and JavaScript?

Answer:

- **JavaScript:** A client-side scripting language mainly used inside browsers.
- **Node.js:** A server-side runtime environment for executing JavaScript outside the browser.

7. Explain the concept of Non-blocking I/O in Node.js.

Answer: Non-blocking I/O means that Node.js can initiate multiple operations without waiting for any to finish. It uses events and callbacks to handle completed tasks asynchronously, resulting in higher efficiency for I/O-bound applications.

8. What are Streams in Node.js?

Answer: Streams are objects that enable reading or writing data continuously. Instead of loading entire files into memory, streams handle data in chunks. Types include:

- Readable
- Writable
- Duplex (both readable and writable)
- Transform (modify data while reading or writing)

9. What is the difference between process.nextTick() and setImmediate()?

Answer:

- **process.nextTick():** Executes the callback before any I/O events, immediately after the current operation completes.
- **setImmediate():** Executes the callback on the next iteration of the event loop, after I/O events are processed.

Pro Tip: Overuse of process.nextTick() can cause starvation of I/O, blocking important asynchronous operations.

10. How does Node.js handle child processes?

Answer: Node.js uses the child_process module to spawn new system processes. These child processes can communicate with the parent via standard input/output streams and can be used for parallel execution to improve performance.

11. What is the REPL in Node.js?

Answer: REPL stands for Read, Eval, Print, and Loop. It is an interactive shell provided by Node.js that processes user input, evaluates it, outputs the result, and loops again. It is useful for testing simple snippets and debugging.

12. What are Global Objects in Node.js?

Answer: Global objects are available throughout a Node.js application without the need to import modules. Common examples include:

- __dirname – Directory name of the current module
- __filename – Filename of the current module
- global – Global namespace object
- process – Information and control about the current Node.js process

13. What is a Callback function in Node.js?

Answer: A callback function is a function passed as an argument to another function, to be called once the parent function completes. Callbacks are essential for handling asynchronous operations in Node.js.

14. What are Promises in Node.js?

Answer: A Promise represents a value that may be available now, later, or never. It simplifies chaining asynchronous operations and avoids callback hell by providing then and catch methods.

15. How can you avoid callback hell in Node.js?

Answer:

- Use Promises instead of nested callbacks
- Use async/await syntax for cleaner, more synchronous-looking code
- Split complex logic into smaller reusable functions
- Leverage control flow libraries like async.js

Related Read: Top 10 Programming Languages to Learn in 2026

16. What is the EventEmitter class in Node.js?

Answer: EventEmitter is a core module that facilitates the implementation of event-driven architecture. It allows emitting and handling events through listener functions using on and emit methods.

17. How do you create a simple server in Node.js?

Answer:
Using the built-in http module:

```
const http = require('http');
const server = http.createServer((req, res) =>  );
server.listen(3000, () => console.log('Server running on port 3000'));
```

18. What is the use of streams over buffers?

Answer: Streams are more memory-efficient because they process large data chunk-by-chunk rather than loading it all into memory at once, making them ideal for working with large files and real-time data processing.

19. What is the purpose of Buffers in Node.js?

Answer: Buffers are used to represent a fixed-size chunk of memory allocated outside the V8 engine. They are primarily used for binary data manipulation, such as when reading from a file or receiving packets over a network.

20. What is the use of process.env in Node.js?

Answer: process.env allows you to access environment variables in a Node.js application, commonly used for sensitive configurations like API keys, database URIs, and environment settings.

21. What is middleware in Express.js?

Answer: Middleware are functions that execute during the lifecycle of a request to the server. Each middleware function can modify the request and response objects or end the request-response cycle or call the next middleware in the stack.

22. How do you handle exceptions in Node.js?

Answer:

- Use try-catch blocks for synchronous code.
- Use .catch() or async/await with try-catch for asynchronous code.
- Use process.on('uncaughtException') and process.on('unhandledRejection') events for global error handling.

Read Also: Most Asked AEM Interview Questions

23. How can you improve Node.js application performance?

Answer:

- Use asynchronous APIs wherever possible
- Implement caching strategies
- Optimize database queries
- Use load balancers and clustering
- Minimize dependencies
- Monitor and optimize the event loop lag

24. What are template engines in Node.js?

Answer: Template engines generate dynamic HTML pages on the server side. Popular Node.js template engines include:

- EJS (Embedded JavaScript Templates)
- Pug (formerly Jade)
- Handlebars

Related Read: Front End Developer Interview Questions and Answers

25. What is the role of package.json in Node.js?

Answer: package.json is the metadata file for any Node.js project. It defines project dependencies, scripts, versions, licensing information, and helps manage project configurations easily through npm.

26. What are CommonJS Modules in Node.js?

Answer: CommonJS is the module system used by Node.js by default. Modules are loaded using require() and exported using module.exports. Each module in Node.js has its own scope.

27. What is the difference between exports and module.exports?

Answer: exports is a shorthand reference to module.exports. If you assign a new object to exports, it will not affect module.exports unless you explicitly reassign it. Always use module.exports for consistency.

28. What are the phases of the Event Loop in Node.js?

Answer: The phases are:

- Timers
- Pending callbacks
- Idle, prepare
- Poll
- Check
- Close callbacks

Each phase has a FIFO queue of callbacks to execute.

29. How is the ‘this’ keyword handled in Node.js modules?

Answer: Inside a Node.js module, this refers to module.exports, not the global object. However, inside a function declared within the module, this refers to the global object.

30. What is middleware chaining in Express.js?

Answer: Middleware chaining happens when multiple middleware functions are executed sequentially. After one middleware completes, it calls next() to pass control to the next middleware in the stack.

31. What is the purpose of process.env.NODE_ENV?

Answer: process.env.NODE_ENV is used to distinguish between environments like development, testing, and production. This enables loading environment-specific configurations.

32. How can you share data between Node.js modules?

Answer: You can export variables, objects, or functions using module.exports and then require them in other files where needed.

33. What are the differences between spawn and fork methods in Node.js?

Answer:

- spawn() launches a new process with a given command. Used for non-Node.js processes.
- fork() creates a new Node.js process specifically designed for IPC (Inter-Process Communication) with the parent process.

34. What is clustering in Node.js?

Answer: Clustering allows Node.js to create multiple child processes (workers) sharing the same server port, leveraging multi-core systems for better scalability and performance.

Related Read: Front End Developer Interview Questions and Answers [[curent_year]]

35. How can you secure a Node.js application?

Answer: Security best practices:

- Use HTTPS
- Sanitize user input
- Store secrets securely (environment variables, vaults)
- Keep dependencies updated
- Use security middleware like Helmet in Express.js

36. What is the difference between process.exit() and process.kill()?

Answer:

- process.exit() terminates the Node.js process immediately with an optional exit code.
- process.kill() sends a signal to another process to terminate it.

37. What are the advantages of using streams instead of buffers?

Answer: Streams are memory-efficient and allow processing data piece-by-piece. They are particularly useful for handling large amounts of data, such as reading files or streaming video.

38. Explain module caching in Node.js.

Answer: When a module is required for the first time, Node.js caches it. Any subsequent require() call returns the same cached instance, improving performance and reducing I/O.

39. What is API rate limiting and how is it implemented in Node.js?

Answer: API rate limiting restricts the number of requests a client can make to an API within a time frame. Libraries like express-rate-limit are commonly used to implement this in Express apps.

40. What is the purpose of async/await in Node.js?

Answer: async/await is syntactic sugar over Promises. It allows writing asynchronous code in a synchronous-looking manner, making it easier to read and maintain.

41. How can you handle file uploads in Node.js?

Answer: Libraries like multer are used in Node.js to handle multipart/form-data, which is primarily used for uploading files.

42. What is CORS and how do you manage it in Node.js?

Answer: CORS (Cross-Origin Resource Sharing) is a security feature that restricts resource sharing between different origins. In Express.js, it can be managed easily using the cors middleware.

43. What are the differences between PUT and PATCH methods in REST APIs?

Answer:

- **PUT:** Replaces an entire resource.
- **PATCH:** Partially updates a resource.

44. How do you implement session management in Node.js?

Answer: Session management is implemented using libraries like express-session. Sessions can be stored in-memory, in databases like Redis, or using secure cookies.

Related Read: Full Stack Developer Skills You Can Add in Your Resume

45. How would you monitor a Node.js application in production?

Answer: Tools like PM2, New Relic, or Datadog are used to monitor Node.js applications. They help track performance metrics, memory usage, error rates, and uptime.

46. What is Helmet in Node.js?

Answer: Helmet is a middleware package for Express.js that sets various HTTP headers to secure your app from common vulnerabilities like cross-site scripting and clickjacking.

47. What is Cross-Site Scripting (XSS) and how can you prevent it in Node.js?

Answer: XSS is an attack where malicious scripts are injected into trusted websites. Prevention involves sanitizing user input, escaping output, and using security libraries like Helmet and DOMPurify.

48. How would you handle asynchronous errors in Express.js routes?

Answer: Wrap async route handlers inside try-catch blocks, or use libraries like express-async-errors to automatically catch async errors and pass them to Express's error handling middleware.

49. What is the advantage of using Redis with Node.js?

Answer: Redis provides a fast, in-memory key-value store, making it ideal for caching, session management, real-time analytics, and message brokering with Node.js.

50. What is the process of deploying a Node.js application?

Answer:

- Prepare production build
- Set environment variables
- Use process managers like PM2
- Reverse proxy with Nginx or Apache
- Deploy to servers like AWS EC2, DigitalOcean, or Heroku

Node.js Advanced Interview Questions

51. What is the difference between synchronous and asynchronous code in Node.js?

Answer:

- **Synchronous:** Blocks the execution until the current operation finishes.
- **Asynchronous:** Executes without blocking; uses callbacks, promises, or async/await to handle results later.

52. What is the use of process.nextTick()?

Answer: process.nextTick() defers the execution of a function until after the current operation completes but before any I/O operations. It is used to prioritize certain tasks.

53. What are Worker Threads in Node.js?

Answer: Worker Threads allow running JavaScript in parallel on multiple threads. Useful for CPU-intensive tasks without blocking the main event loop.

54. How does load balancing work in Node.js applications?

Answer: Load balancing can be achieved by clustering multiple Node.js instances across CPU cores, and externally through load balancers like Nginx or AWS Elastic Load Balancing.

Related Read: HTML and CSS Interview Questions & Answers [current year]

55. What is the difference between global and local installation of npm packages?

Answer:

- **Global installation (-g):** Makes the package available system-wide, usually for CLI tools.
- **Local installation:** Installs the package inside node_modules of the project for project-specific usage.

56. What is the HTTP2 module in Node.js?

Answer: The HTTP2 module allows Node.js to support HTTP/2 protocol, enabling faster communication through multiplexing, header compression, and server push.

57. What is middleware error handling in Express.js?

Answer: Error-handling middleware functions have four arguments (err, req, res, next) and handle errors by sending appropriate responses or delegating them further.

58. How can you create your own EventEmitter in Node.js?

Answer:

```
const EventEmitter = require('events');
const emitter = new EventEmitter();
emitter.on('event', () => console.log('An event occurred!'));
emitter.emit('event');
```

59. What is the difference between readFile and createReadStream in Node.js?

Answer:

- readFile: Reads the entire file into memory before processing it.
- createReadStream: Reads the file chunk-by-chunk using streams, making it memory-efficient for large files.

60. What is the role of libuv in Node.js?

Answer: libuv is a multi-platform C library that Node.js uses to handle asynchronous I/O operations through its event-driven architecture. It abstracts non-blocking I/O across platforms.

61. What is process.on('exit') in Node.js?

Answer: process.on('exit', callback) registers a callback that executes when the Node.js process is about to exit, allowing cleanup tasks before shutdown.

62. What are Signals in Node.js?

Answer: Signals are a limited form of inter-process communication used to notify a process that a specific event occurred. Example: SIGINT for Ctrl+C termination.

63. What is a Zombie Process? Can it occur in Node.js?

Answer: A zombie process is a process that has completed execution but still has an entry in the process table. While rare in Node.js, improper child process handling could theoretically lead to zombies.

64. What is Server Push in HTTP/2?

Answer: Server Push allows servers to send resources to clients proactively without waiting for explicit requests, improving page load times by preloading resources.

Related Read: Must-Have Resume Skills for a Tech Lead Role in 2026

65. What is TLS/SSL in Node.js?

Answer: TLS/SSL encrypts data between the server and the client to ensure secure transmission. Node.js has a built-in tls module to implement secure sockets.

66. How does garbage collection work in Node.js?

Answer: Node.js relies on V8's garbage collector to automatically reclaim memory occupied by objects no longer in use, helping prevent memory leaks.

67. What is the async_hooks module?

Answer: The async_hooks module provides an API to track asynchronous resources in Node.js, useful for performance monitoring and debugging.

68. What is DNS module in Node.js?

Answer: The dns module provides functions to perform DNS lookup and name resolution functions, like resolving domain names to IP addresses asynchronously.

69. What is the cluster module in Node.js?

Answer: The cluster module enables creation of child processes (workers) that all share the same server port, allowing full CPU core utilization in Node.js applications.

70. What is the difference between fork() and spawn() methods?

Answer:

- spawn(): Launches a new process with a given command.
- fork(): Specifically spawns a Node.js process and sets up IPC communication channel with the parent.

71. What are sticky sessions in Node.js clustering?

Answer: Sticky sessions ensure that the same user is always routed to the same server worker based on their session ID, which is essential for maintaining session persistence in load-balanced applications.

72. How can you debug a Node.js application?

Answer:

- Using built-in --inspect flag with Chrome DevTools

- Using IDE debuggers (like VSCode)
- Logging strategically with libraries like Winston

73. What is npm audit?

Answer: npm audit is a command that performs a security audit of project dependencies and highlights known vulnerabilities that should be addressed.

74. What are peer dependencies in npm?

Answer: Peer dependencies specify that a package expects another package to be installed alongside it, but does not install it automatically. Useful in plugin systems.

Related Read: Tips to Write Software Engineer Resume [+Samples]

75. What are cyclic dependencies in Node.js and how to avoid them?

Answer: Cyclic dependencies occur when two or more modules depend on each other circularly. They can cause unpredictable behavior. To avoid them, refactor shared logic into a separate module.

Node.js Scenario-Based and Practical Questions

76. How would you handle a sudden spike in traffic in a Node.js application?

Answer: Scale horizontally using clustering and load balancers, implement caching strategies with Redis, optimize database queries, and use CDNs for static content delivery.

77. How would you implement logging in a Node.js app?

Answer: Use libraries like Winston, Morgan, or Bunyan. Log critical information with timestamps, and separate logs by severity levels (info, warn, error).

78. How would you secure sensitive data like API keys in a Node.js project?

Answer: Store sensitive data in environment variables using dotenv files (.env), and never hardcode credentials in the source code.

79. How would you design a RESTful API using Node.js?

Answer: Use Express.js to define routes, separate controllers, services, and models, follow HTTP methods appropriately (GET, POST, PUT, DELETE), and structure resources logically.

80. How would you handle file uploads securely in Node.js?

Answer: Use multer for handling uploads, validate file types, limit file sizes, store uploads in secure directories, and scan files for malware if needed.

81. How can you schedule background tasks in Node.js?

Answer: Use libraries like node-cron or Agenda.js to schedule recurring or time-based tasks like email notifications, report generation, or database cleanups.

82. How would you implement WebSocket communication in Node.js?

Answer: Use the ws library or frameworks like Socket.IO for real-time, bi-directional communication between the client and server.

83. How would you design a chat server using Node.js?

Answer: Use Socket.IO to manage rooms, broadcast messages, store chat history in a database like MongoDB, and implement authentication for users.

84. How would you implement API versioning in Node.js?

Answer: Organize routes based on versions (e.g., /api/v1/, /api/v2/) and separate controllers for each version to avoid breaking existing API clients.

85. How would you optimize a Node.js app for performance?

Answer:

- Use asynchronous code
- Implement caching (Redis, CDN)
- Use clustering
- Optimize database queries
- Compress HTTP responses (gzip)
- Use monitoring tools to detect bottlenecks

Node.js Interview Questions for Freshers

86. What are the key features of Node.js?

Answer: Event-driven, asynchronous, single-threaded, scalable, and built on the V8 JavaScript engine.

87. What is the role of package-lock.json?

Answer: package-lock.json locks the dependency tree, ensuring consistent installations across different environments.

88. What is an HTTP module in Node.js?

Answer: The HTTP module allows Node.js to create HTTP servers and handle client requests and responses natively.

89. What is the use of util module in Node.js?

Answer: The util module provides helpful utility functions like promisify() for converting callback-based functions into promises.

90. How does require() work in Node.js?

Answer: require() reads and executes a JavaScript file and returns the exports object, making it available for use in other modules.

Related Read: Teradata Interview Questions and Answers [current_year]

Node.js Interview Questions for Experienced Professionals

91. How would you implement rate limiting in a Node.js API?

Answer: Use express-rate-limit middleware to set limits based on IP addresses and define time windows for API usage.

92. How would you manage environment variables securely in production?

Answer: Use encrypted environment variable management services like AWS Secrets Manager, Vault by HashiCorp, or environment-specific .env files.

93. How would you use Redis in a Node.js project?

Answer: Redis can be used for caching, session storage, message queues, and real-time analytics in a Node.js application.

94. How would you protect an Express API from brute-force attacks?

Answer: Implement rate-limiting middleware, IP blacklisting, CAPTCHA challenges, and account lockout mechanisms after repeated failed attempts.

Related Read: React and next.js Interview Questions and Answers [current_year]

95. How would you implement global error handling in Node.js?

Answer: Create a centralized error-handling middleware, handle unhandled promise rejections with process.on('unhandledRejection'), and manage uncaught exceptions with process.on('uncaughtException').

96. How would you implement JWT authentication in Node.js?

Answer: Use libraries like jsonwebtoken to sign tokens on login and verify them on protected routes in the Express middleware.

97. How do you perform load testing on a Node.js application?

Answer: Use tools like Artillery, k6, or Apache JMeter to simulate high traffic and evaluate the application's performance under load.

98. How do you handle race conditions in Node.js?

Answer: Handle shared resources carefully, use database-level locks or optimistic concurrency control strategies, and design code flow predictably.

99. What is Event Loop Starvation in Node.js?

Answer: Event loop starvation happens when heavy synchronous operations block the event loop, preventing the processing of pending events, leading to application slowdowns or freezes.

100. How would you optimize memory usage in a Node.js application?

Answer:

- Use streams for large data
- Release unused references
- Profile and analyze memory leaks with Chrome DevTools
- Apply batching or throttling techniques for heavy processing

Related Read: Software Engineer Interview Questions and Answers [[curent_year]]

FAQs on Node.js Interview Questions and Answers in 2026

1. What is Node.js mainly used for?
Node.js is mainly used for building scalable server-side and networking applications, especially APIs, real-time apps, microservices, and data-intensive applications.

2. Is Node.js good for building APIs?
Yes, Node.js is excellent for building fast, scalable REST APIs, GraphQL APIs, and microservices, due to its non-blocking, event-driven architecture.

3. What is Event Loop in Node.js with example?
The Event Loop in Node.js handles asynchronous callbacks. For example, after an I/O operation like file reading completes, its callback is executed in the event loop.

4. What is the difference between Node.js and Express.js?
Node.js is the runtime environment for executing JavaScript on the server-side, while Express.js is a web application framework built on top of Node.js to simplify routing and middleware handling.

5. How to handle authentication in a Node.js application?
Authentication can be handled using Passport.js for strategy-based authentication or JSON Web Tokens (JWT) for stateless authentication.

6. Is Node.js single-threaded or multi-threaded?
Node.js is single-threaded for event handling but can create background threads internally for I/O operations through libuv.

7. What is npm and why is it important?
npm (Node Package Manager) is crucial for managing JavaScript libraries, dependencies, and tools efficiently in any Node.js project.

8. Can Node.js be used for enterprise applications?
Yes. Companies like Netflix, PayPal, and LinkedIn use Node.js for their enterprise-grade systems due to its scalability, performance, and active ecosystem.